



Uema
UNIVERSIDADE ESTADUAL
DO MARANHÃO

Visão Computacional

Aula 10 – Detecção de movimento

Prof. Dr. Lúcio Flávio de A Campos



LabVIR
Laboratório de Visão Computacional,
Inteligência e Robótica - Uema

Roteiro da aula de hoje

Diferenciação de Frames

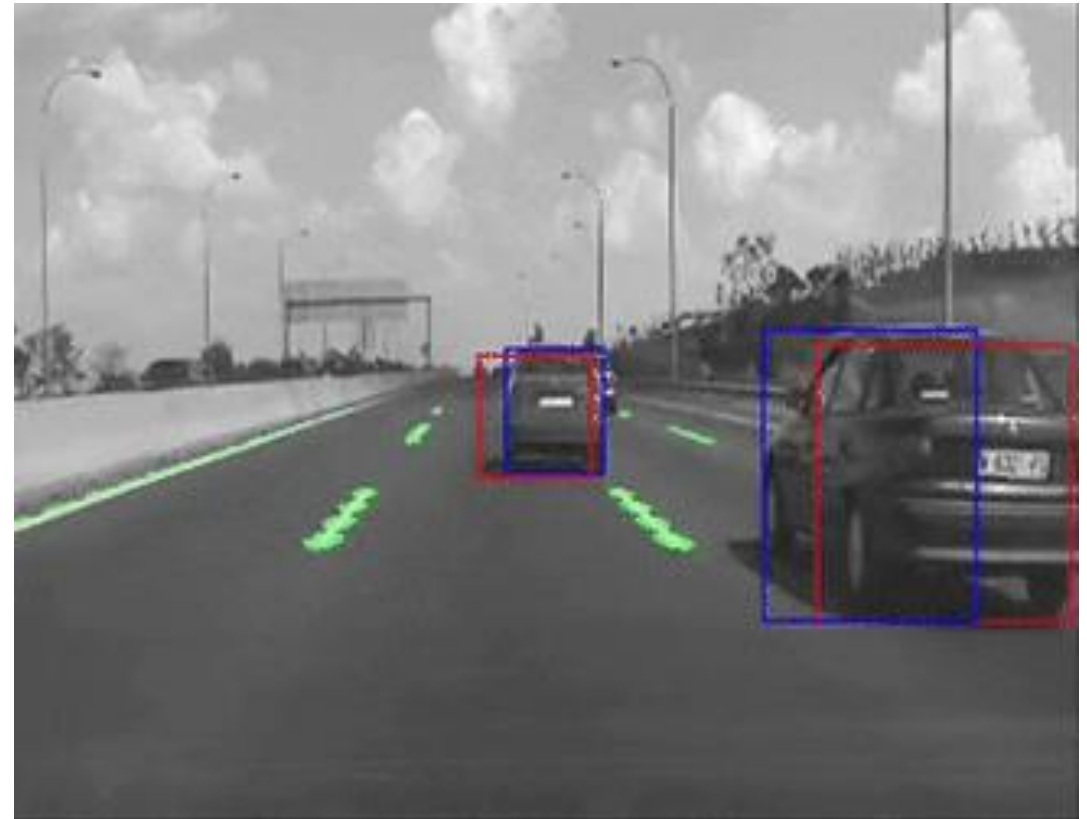
Fluxo óptico

Subtração de Background

Detecção de Objetos

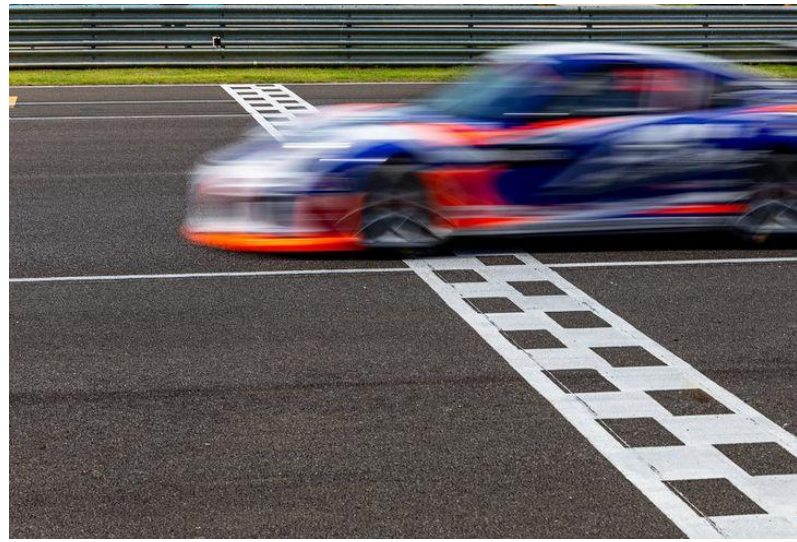
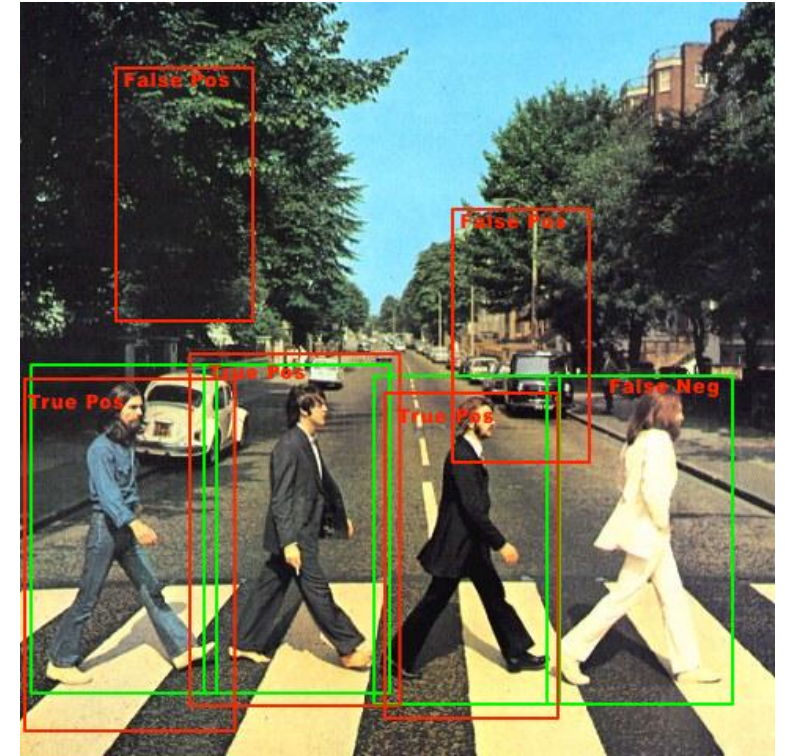
Detecção de movimento

- Medir movimento relativo de objetos
- Rastreo
- Estimar a posição de um objeto
- Etapas:
 - Captura de vídeo
 - Detecção do objeto
 - Classificação do objeto
 - Rastreo do objeto



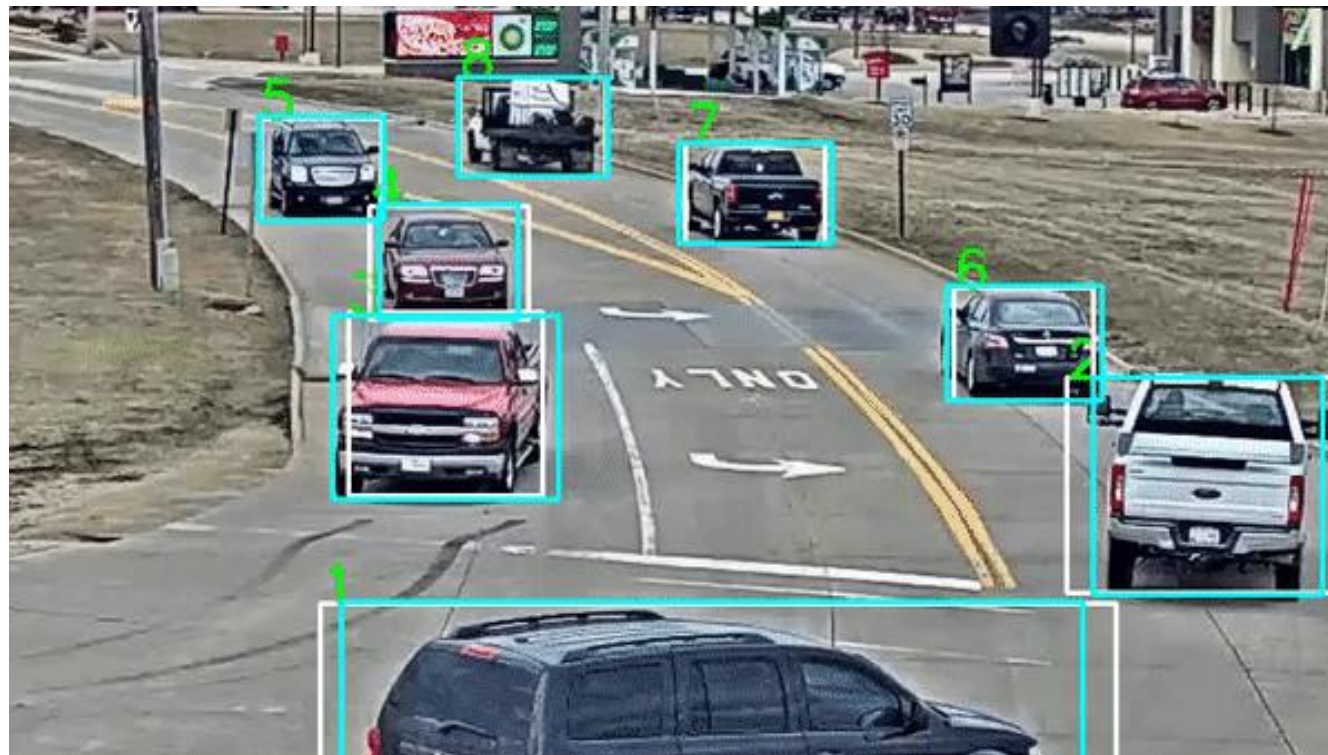
Detecção de objetos - Desafios

- Imagens ocultas
- Imagens borradas (em movimento)
- Forma do objeto
- Falsos Positivos



Técnicas para rastreamento de objetos

- Diferenciação de frames
- Subtração de backgrounds
- Fluxo óptico



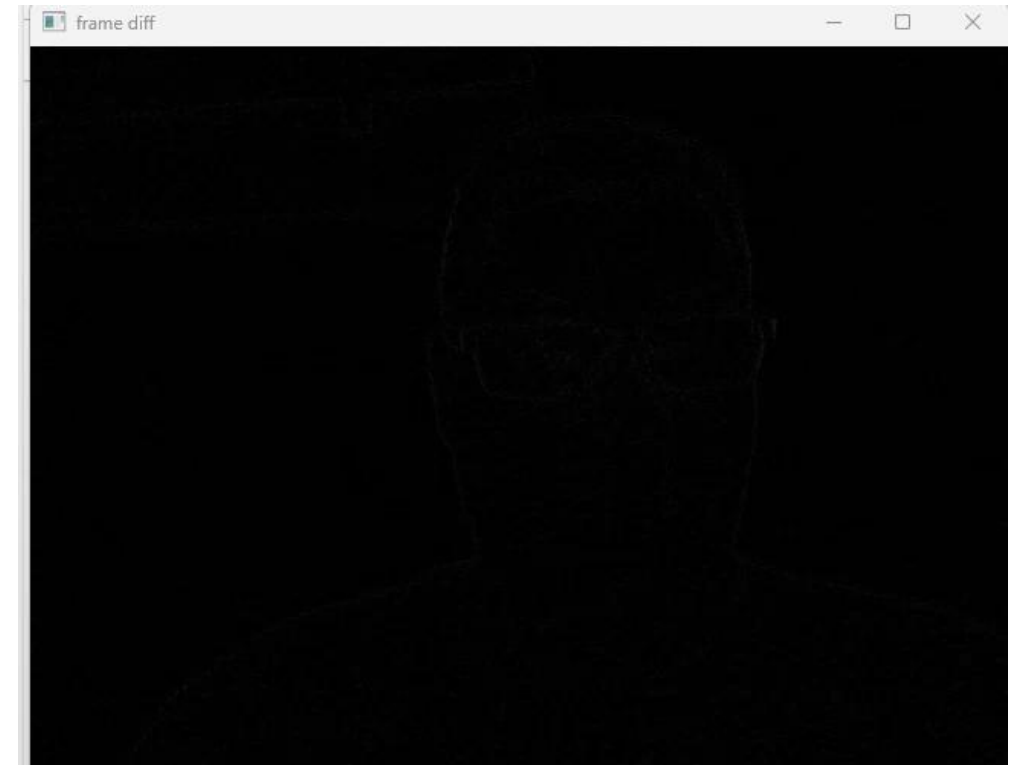
Técnicas para rastreamento de objetos – Diferenciação de Frames

- Trata de determinar a presença de objetos em movimento, calculando a diferença de pixels entre duas imagens consecutivas em um vídeo. Embora o cálculo geral seja simples e fácil de implementar, a complexidade aumenta por causa do objeto "em movimento", o que pode comprometer a precisão.
- As técnicas de diferenciação de frames têm alta precisão e tempo computacional relativamente baixo ou moderado

Função `cv2.absdiff(src1, src2)`

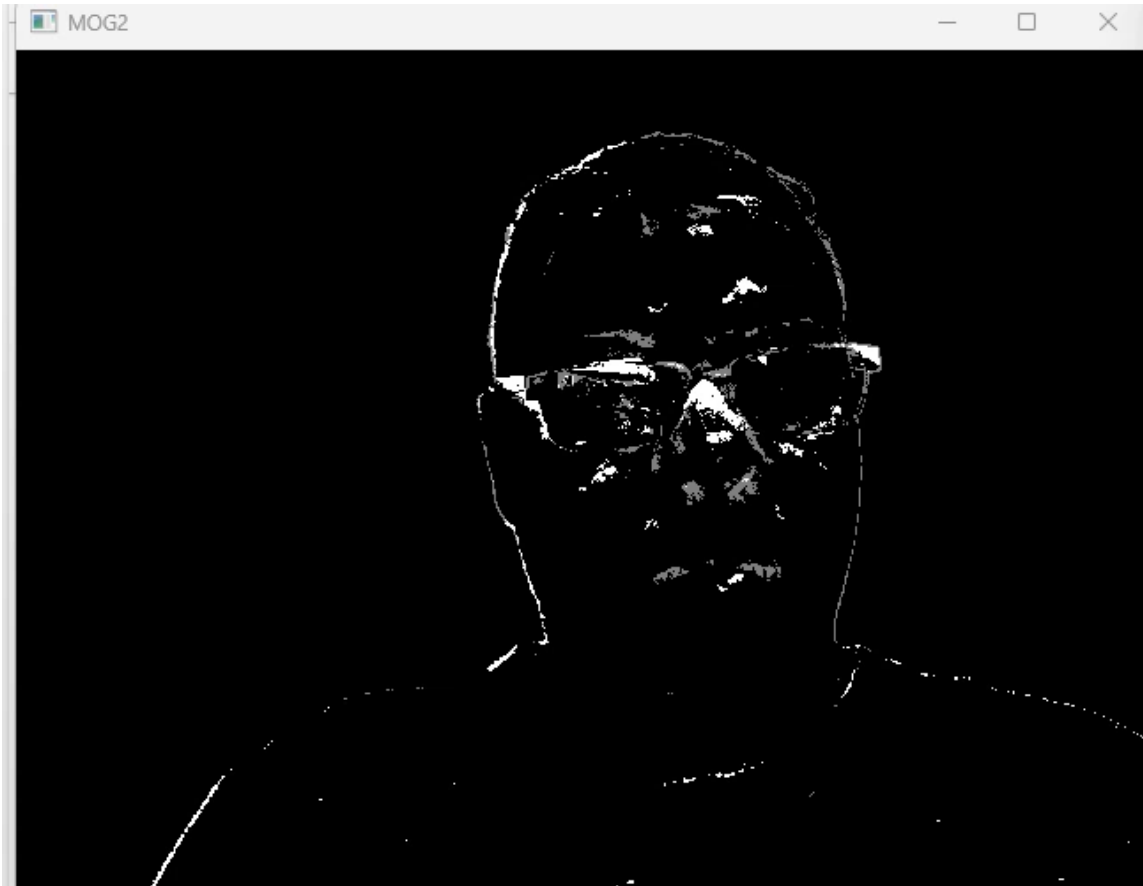
- `src1` - primeira matriz de entrada ou um escalar.
- `src2` – Segunda matriz de entrada ou um escalar.

Funciona bem para fundos estáticos



Técnicas para rastreamento de objetos – Subtração de Background

- Separar o fundo do primeiro plano em fluxos de vídeo.
- Todos os métodos de subtração de fundo são moderados em precisão, bem como tempo computacional.



Técnicas para rastreamento de objetos – Subtração de Background

Funções:

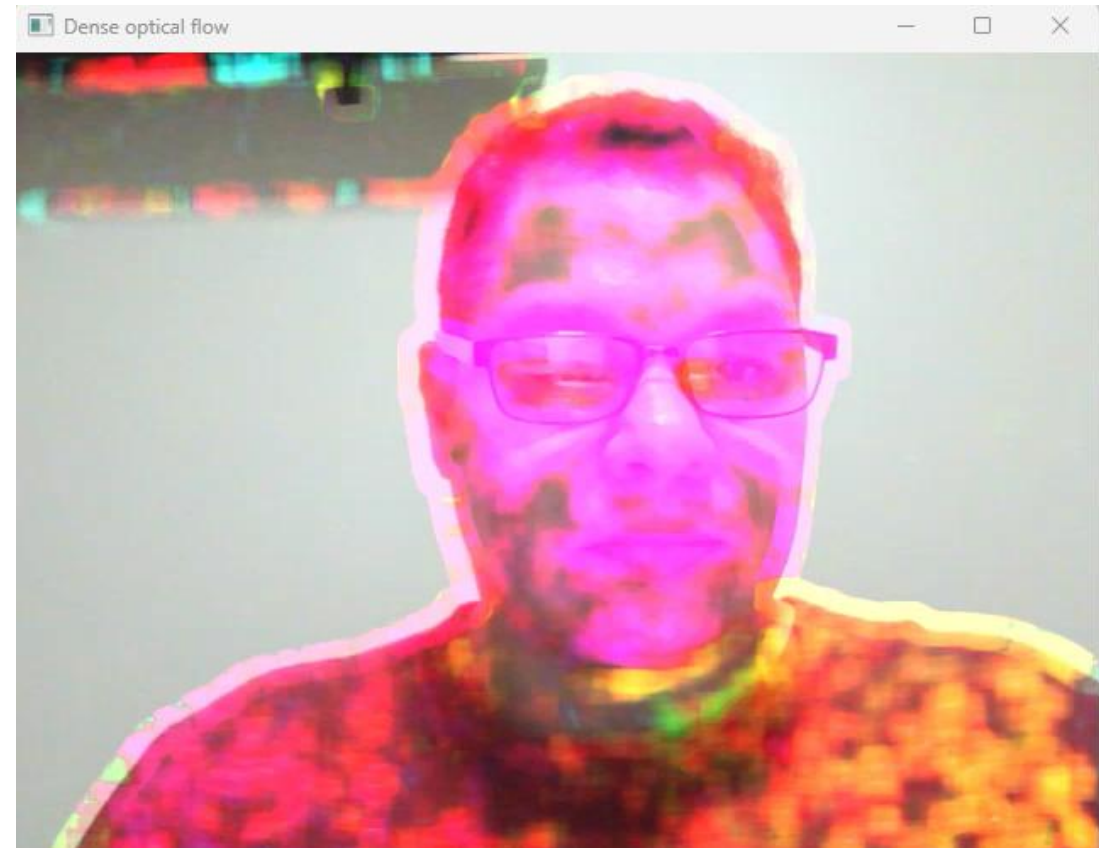
- `createBackgroundSubtractorMOG2(history, varThreshold, detectShadows)`
- `createBackgroundSubtractorKNN (history, dist2Threshold, detectShadows)`

Sendo:

- **history** = é o número de frame usados para construir o modelo estatístico do plano de fundo. Quanto menor for o valor, mais rápidas as alterações no plano de fundo serão consideradas pelo modelo e, portanto, serão consideradas como plano de fundo.
- **dist2Threshold** = Limiar na distância quadrada entre o pixel e a amostra para decidir se um pixel está próximo dessa amostra.
- **varThreshold** = Limiar na distância quadrada de Mahalanobis entre o pixel e o modelo para decidir se um pixel está bem descrito pelo modelo de fundo. Este parâmetro não afeta a atualização em segundo plano.
- **detectShadows** = Se setado como True, as sombras serão apresentadas na imagem.

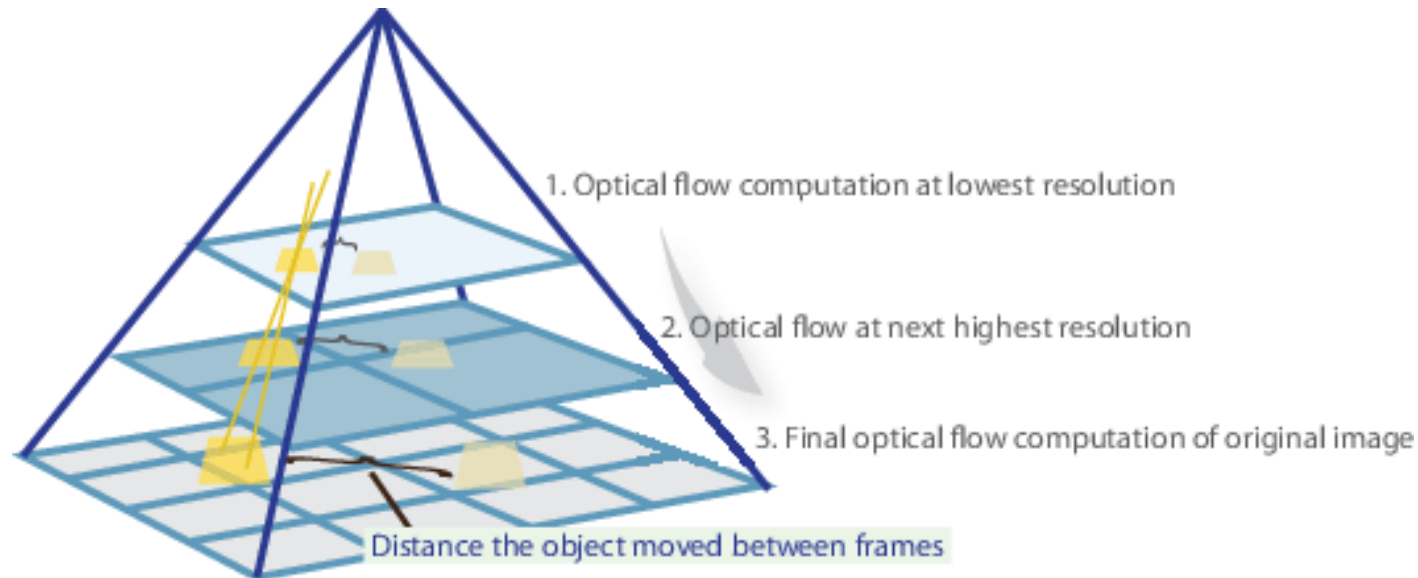
Técnicas para rastreamento de objetos – Fluxo Óptico

- Analisa o movimento dos pixels de um frame para outro.
- É particularmente útil onde você tem movimento de fundo que não permite o uso de outros métodos.
- É um dos algoritmos de visão computacional mais rápidos de configurar, embora seja bastante custoso em termos computacionais.



Técnicas para rastreamento de objetos – Fluxo Óptico

- Algoritmo Farneback
 - Gera uma pirâmide de imagens, onde cada nível possui uma resolução inferior em relação ao nível anterior.
 - Quando um nível de pirâmide maior que 1 é selecionado, o algoritmo pode rastrear os pontos em vários níveis de resolução, começando no nível mais baixo.
 - Aumentar o número de níveis de pirâmide permite que o algoritmo trate de maiores deslocamentos de pontos entre quadros. No entanto, o número de cálculos também aumenta.



Técnicas para rastreamento de objetos – Fluxo Óptico

Função `calcOpticalFlowFarneback()`

`cv2.calcOpticalFlowFarneback(prev, next, flow, pyr_scale, levels, winsize, iterations, poly_n, poly_sigma, flags)`

• Parâmetros:

- **prev** = frame de entrada
- **next** = segundo frame do mesmo tamanho que o **prev**
- **flow** = computa o fluxo da imagem que tem o mesmo tamanho do **prev**
- **pyr_scale** = parâmetro, especificando a escala da imagem (<1) para construir pirâmides para cada imagem; $\text{pyr_scale} = 0.5$ significa uma pirâmide clássica, onde cada camada seguinte é duas vezes menor que a anterior.
- **levels** = número de camadas da pirâmide incluindo a imagem inicial; $\text{níveis} = 1$ significa que nenhuma camada extra é criada e apenas as imagens originais são usadas.
- **winsize** = tamanho médio da janela; valores maiores aumentam a robustez do algoritmo para ruído de imagem e fornecem mais chances de detecção de movimento rápido, mas geram mais campo de movimento borrado.

Técnicas para rastreamento de objetos – Fluxo Óptico

Função `calcOpticalFlowFarneback()`

`cv2.calcOpticalFlowFarneback(prev, next, flow, pyr_scale, levels, winsize, iterations, poly_n, poly_sigma, flags)`

•Parâmetros:

- iterations** = número de interações do algoritmo de cada level da pirâmide.
- **poly_n** = tamanho da vizinhança do pixel usado para encontrar a expansão polinomial em cada pixel; valores maiores significam que a imagem será aproximada com superfícies mais suaves, gerando algoritmo mais robusto e campo de movimento mais borrado, normalmente `poly_n = 5` ou `7`.
- poly_sigma** = desvio padrão do Gaussiano que é usado para suavizar as derivadas usadas como base para a expansão polinomial; para `poly_n = 5`, você pode definir `poly_sigma = 1,1`, para `poly_n = 7`, um bom valor seria `poly_sigma = 1,5`.
- flags** = operações de fluxo
 1. **OPTFLOW_USE_INITIAL_FLOW** usa o fluxo de entrada como uma aproximação de fluxo inicial.
 2. **OPTFLOW_FARNEBACK_GAUSSIAN** usa o filtro Gaussiano do tamanho da janela em vez de um filtro de caixa do mesmo tamanho para estimativa de fluxo óptico; geralmente, esta opção fornece um fluxo mais preciso do que com um filtro de caixa, ao custo de uma velocidade menor; normalmente, o tamanho da caixa para uma janela Gaussiana deve ser definido com um valor maior para atingir o mesmo nível de robustez.